Manual técnico de MetaFit

Servicio Nacional de Aprendizaje (SENA),

Actividades de documentación y entrega de software en procesos de implantación

Instructor:Yara Amaya Javier Emilio

12 de octubre de 2026

Manual técnico de MetaFit

Manual técnico de MetaFit v2.0, el sistema de gestión deportiva de Sport Gym Sede 80 (Bogotá): explica cómo configurar, desplegar y mantener el backend, el panel web, la app móvil y la base de datos.

1. Portada y control del documento

Tabla 1
Datos de identificación del documento

Campo	Contenido
Nombre del proyecto	MetaFit, sistema de gestión deportiva
Versión del software	1.0
Versión del manual	1.0
Fecha	12/10/2026
Autor(es)	Isabella Caldas Perez y Equipo MetaFit (SENA)
Institución	Servicio Nacional de Aprendizaje (SENA),
Curso	Actividades de documentación y entrega de software en procesos de implantación
Instructor	Yara Amaya Javier Emilio
Cliente / Organización	Sport Gym Sede 80, Bogotá, Colombia
Estado	Borrador

Historial de cambios

Tabla 2
Historial de cambios del manual

Versión	Fecha	Autor	Descripción del cambio
1.0	12/10/2026	Juan Sebastián Carvajal	Versión inicial del manual técnico

Aprobaciones

Tabla 3
Aprobaciones del manual

Nombre	Cargo	Firma	Fecha

2. Índice

1. Portada y control del documento **2**
Historial de cambios 2
Aprobaciones 2
2. Índice **3**
3. Introducción **3**
3.1 Propósito del manual 3
3.2 Alcance 3
3.3 Público objetivo 3
3.4 Documentos relacionados 3
3.5 Convenciones del documento 3
4. Descripción general del sistema **4**
4.1 Objetivo y funcionalidades principales 4
4.2 Módulos y componentes 4
4.3 Tecnologías utilizadas 4
4.4 Ficha técnica del sistema 5
4.5 Casos de uso 6
4.6 Historias de usuario 6
5. Arquitectura del sistema **7**
5.1 Diagrama de arquitectura 7
5.2 Diagrama de despliegue 8
5.3 Modelo de datos 9
5.4 Flujo de información 10
6. Requisitos del entorno **11**
6.1 Requisitos de hardware 11
6.2 Requisitos de software 11
6.3 Dependencias externas 11
7. Configuración **12**
7.1 Archivos de configuración 12
7.2 Variables de entorno y parámetros 12
7.3 Usuarios, roles y permisos 13
8. Estructura del código fuente **13**
8.1 Organización de carpetas 13
8.2 Estándares de codificación 14
8.3 Componentes y clases principales 14
9. Base de datos **15**
9.1 Motor y versión 15
9.2 Scripts de creación y migración 15
9.3 Diccionario de datos 15
10. Interfaces e integraciones (APIs) **16**

11. Seguridad **17**
11.1 Autenticación y autorización 17
11.2 Protección de datos y cifrado 18
11.3 Gestión de registros (logs) y auditoría 18
11.4 Cumplimiento normativo 18
12. Pruebas **18**
12.1 Plan de pruebas 18
12.2 Casos de prueba y resultados 19
12.3 Incidencias conocidas 19
13. Despliegue y entrega **19**
13.1 Entornos 19
13.2 Procedimiento de despliegue 19
13.3 Plan de contingencia y retorno (rollback) 20
13.4 Lista de entregables 20
14. Mantenimiento y soporte **20**
14.1 Tareas de mantenimiento periódico 20
14.2 Monitorización y rendimiento 21
14.3 Gestión de incidencias y niveles de soporte 21
14.4 Procedimiento de actualización 21
14.5 Contactos de soporte 21
15. Solución de problemas (FAQ técnica) **22**
16. Glosario **22**
17. Anexos **24**

3. Introducción

3.1 Propósito del manual

Este manual permite que un técnico ajeno al proyecto configure, despliegue y mantenga MetaFit sin ayuda del equipo desarrollador.

3.2 Alcance

Cubre el backend (API REST), el panel web, la app móvil, la base de datos, las automatizaciones (cron y n8n) y el despliegue. No cubre el uso funcional del sistema, que está en el manual de usuario.

3.3 Público objetivo

Administradores de sistemas, desarrolladores que den mantenimiento y personal de soporte técnico de Sport Gym Sede 80.

3.4 Documentos relacionados

Manual de usuario (**`MANUAL_USUARIO.md`**), manual de despliegue (**`manual_despliegue.md`**), manual Postman (**`MANUAL_POSTMAN.md`**), informe de QA (**`QA_REPORT.md`**), diagramas (**`DIAGRAMAS.md`**) y la documentación Swagger en **`/api-docs`**.

3.5 Convenciones del documento

Los comandos y rutas van en formato de código. Los valores entre corchetes, como `[JWT_SECRET]`, se deben sustituir por el valor real. Las rutas son relativas a la raíz del repositorio `Equipo_Metafit` y las tablas de la base de datos se escriben en MAYÚSCULAS.

4. Descripción general del sistema

4.1 Objetivo y funcionalidades principales

MetaFit centraliza la operación de un gimnasio: afiliados, planes de entrenamiento y nutrición, pagos, seguimiento diario (ejercicio, agua y alimentos) y notificaciones. El personal usa un panel web y los afiliados una app móvil, ambos conectados a la misma API REST.

4.2 Módulos y componentes

Tabla 4
Módulos del sistema y responsables

Módulo	Descripción	Responsable
Afiliados	Alta, edición, ciclos, restricciones médicas y foto de perfil	Recepcionista y Administrador
Usuarios y personal	Cuentas del gimnasio y asignación de roles	Administrador
Planes de entrenamiento	Rutinas por ciclo con ejercicios, series y repeticiones	Entrenador
Planes nutricionales	Comidas por ciclo con alimentos y calorías objetivo	Entrenador
Catálogos	CRUD de ejercicios, alimentos y restricciones	Administrador y Entrenador
Pagos y finanzas	Registro de pagos, factura por correo, ingresos y métricas	Recepcionista y Administrador
Seguimiento diario	Registro de ejercicio, agua, alimentos y progreso físico desde el móvil	Afiliado

Módulo	Descripción	Responsable
Dashboard y notificaciones	KPIs del gimnasio y alertas	Administrador
Configuración	Precio de la membresía	Administrador

4.3 Tecnologías utilizadas

Tabla 5
Tecnologías utilizadas por capa

Capa	Tecnología	Versión
Frontend web	React, Vite, react-router-dom, Chart.js, jsPDF	React 19, Vite 6, Router 7
App móvil	React Native con Expo	Expo SDK 55, React Native 0.83
Backend	Node.js, Express, mysql2 (sin ORM)	Node 22, Express 4
Base de datos	MariaDB, compatible con MySQL	11
Servidor / Cloud	Render (backend, Docker), Railway (base de datos), EAS (móvil)	n/a
Servicios externos	Cloudinary (fotos), Brevo (correo), Expo Push, n8n (automatizaciones)	n/a

4.4 Ficha técnica del sistema

Según el Ministerio de Tecnologías de la Información y las Comunicaciones [Ministerio TIC] (2018), un manual técnico debe incluir como mínimo los elementos de la Tabla 6.

Tabla 6
Ficha técnica del sistema

Elemento	Contenido
Nombre del sistema	MetaFit, sistema de gestión deportiva
Sigla	MetaFit
Descripción del sistema	Administra afiliados, planes de entrenamiento y nutrición, pagos, seguimiento diario y notificaciones de Sport Gym Sede 80, con un panel web para el personal y una app móvil para los afiliados
Versión	2.0 (actual y en producción)
Categoría	Sistema misional
Fabricante	Equipo MetaFit (SENA); Encargado:Isabella Caldas
Fecha de vencimiento del soporte	[Por definir: no hay un contrato de soporte documentado]
Sistema operativo	Servidor: contenedor Linux (imagen `node:22-alpine`). Clientes: navegador web y app móvil Expo
Lenguaje de programación	JavaScript: Node.js 22 y Express (backend), React 19 (web), React Native con Expo SDK 55 (móvil); SQL
Plataforma de base de datos	MariaDB 11, compatible con MySQL

Elemento	Contenido
Documentación técnica y funcional	Repositorio **`Equipo_Metafit`** ([URL del repositorio]), carpeta **`documentación`**/ y Swagger en **`/api-docs`**
Arquitectura tecnológica	Web y app móvil (cliente-servidor) sobre una API REST, desplegada en la nube (Render y Railway)

4.5 Casos de uso

Figura 1
Casos de uso por rol



La Figura 1 muestra las funciones de cada rol. Los tres primeros roles usan el panel web y el afiliado usa la app móvil. El detalle de cada función está en las historias de usuario (Tabla 7).

4.6 Historias de usuario

Tabla 7
Historias de usuario

ID	Rol	Historia	Estado
AU-01	Todos	Iniciar sesión con correo y contraseña	Hecha
AU-02	Todos	Recuperar la contraseña por correo electrónico	Hecha
AU-03	Todos	Que la sesión expire a las 8 horas por seguridad	Hecha
AU-04	Sistema	Bloquear el login tras 10 intentos fallidos en 15 minutos	Hecha
AD-01	Administrador	Ver el dashboard con KPIs: total de afiliados, activos, ciclos en curso y restricciones	Hecha
AD-02	Administrador	Gestionar el personal (CRUD de entrenadores y recepcionistas)	Hecha

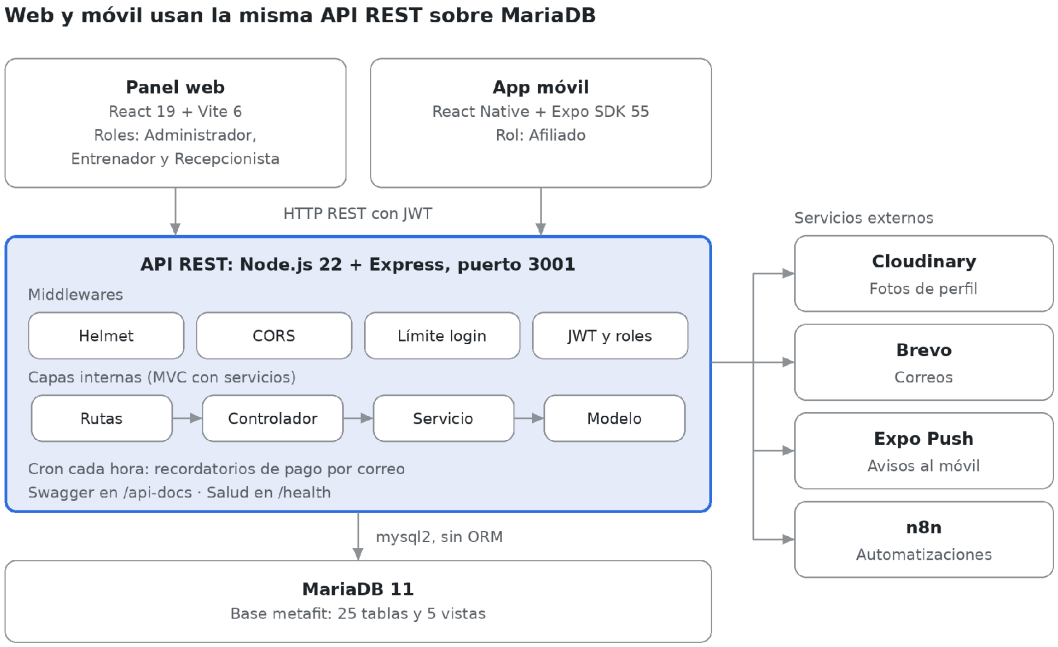
ID	Rol	Historia	Estado
AD-03	Administrador	Ver finanzas: ingresos por mes, por recepcionista y total recaudado	Hecha
AD-04	Administrador	Configurar el precio de la membresía	Hecha
AD-05	Administrador	Acceder a todos los afiliados, planes y catálogos	Hecha
EN-01	Entrenador	Ver la lista de afiliados con perfil, restricciones y ciclo activo	Hecha
EN-02	Entrenador	Crear y gestionar ciclos de entrenamiento	Hecha
EN-03	Entrenador	Crear planes de entrenamiento con rutinas y ejercicios	Hecha
EN-04	Entrenador	Crear planes nutricionales con el detalle de alimentos por comida	Hecha
EN-05	Entrenador	Registrar el progreso físico: peso, % de grasa, medidas e IMC	Hecha
EN-06	Entrenador	Asignar restricciones médicas a los afiliados	Hecha
EN-07	Entrenador	Enviar una notificación push al afiliado al crear un plan	Hecha
RC-01	Recepcionista	Registrar afiliados nuevos (usuario y perfil en una transacción)	Hecha
RC-02	Recepcionista	Editar los datos de afiliados existentes	Hecha
RC-03	Recepcionista	Registrar pagos de membresía	Hecha
RC-04	Recepcionista	Ver el historial de pagos por afiliado	Hecha
RC-05	Recepcionista	Subir la foto de perfil del afiliado	Hecha
AF-01	Afiliado	Ver mi perfil, ciclo activo y restricciones médicas	Hecha
AF-02	Afiliado	Ver mi plan de entrenamiento: rutinas, ejercicios, series y repeticiones	Hecha
AF-03	Afiliado	Ver mi plan nutricional: alimentos por comida y cantidades	Hecha
AF-04	Afiliado	Marcar ejercicios como completados o no completados por día	Hecha
AF-05	Afiliado	Registrar el consumo de agua (vasos por día)	Hecha
AF-06	Afiliado	Registrar el consumo de alimentos del plan	Hecha
AF-07	Afiliado	Ver mi historial de progreso físico con IMC	Hecha
AF-08	Afiliado	Recibir notificaciones push: planes nuevos y recordatorios	Hecha
AF-09	Afiliado	Recuperar la contraseña desde la app	Hecha

5. Arquitectura del sistema

5.1 Diagrama de arquitectura

MetaFit usa una arquitectura cliente-servidor de tres capas. El panel web (React) y la app móvil (Expo) consumen una API REST en Node.js y Express, que accede a MariaDB con mysql2, como muestra la Figura 2. El backend sigue el patrón MVC con capa de servicios: rutas, controladores, servicios y modelos.

Figura 2
Arquitectura general de MetaFit



La API es el único punto de acceso a la base de datos: ni el panel web ni la app móvil la consultan directamente.

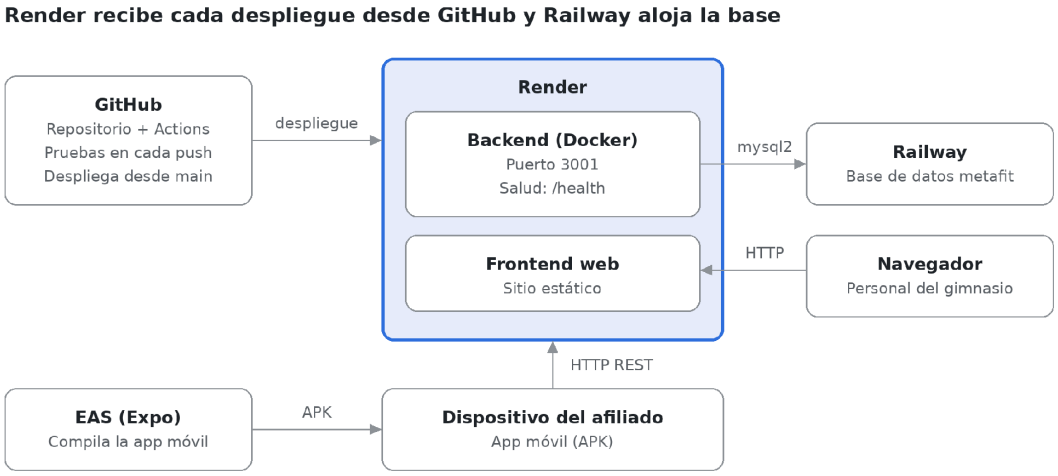
5.2 Diagrama de despliegue

En producción el backend corre en un contenedor Docker en Render (puerto 3001, health check en `/health`), la base de datos en Railway, las fotos en Cloudinary y la app móvil se compila con EAS, como muestra la Figura 3. En local, `docker-compose.yml` levanta los servicios de la Tabla 8.

Tabla 8
Servicios y puertos del entorno local

Servicio	Puerto	URL local
Frontend web	5173	http://localhost:5173
Backend (API)	3001	http://localhost:3001
phpMyAdmin	8080	http://localhost:8080
MariaDB	3306	localhost:3306
n8n	5678	http://localhost:5678

Figura 3
Despliegue de MetaFit en producción



Al llegar a `main`, GitHub Actions despliega el backend y el frontend en Render. La base de datos vive en Railway y la app móvil se compila con EAS.

5.3 Modelo de datos

El modelo tiene 25 tablas y 5 vistas. El núcleo es USUARIO (rol Administrador, Recepcionista, Entrenador o Afiliado), de la que depende AFILIADO y sus CICLO (periodos de entrenamiento). De cada ciclo cuelgan RUTINA, PLAN_NUTRICIONAL, PROGRESO_FISICO y los registros diarios. Los catálogos son EJERCICIO, ALIMENTO y RESTRICCION, relacionados por tablas de exclusión. PAGO se asocia a cada usuario afiliado. La Figura 4 resume las entidades principales.

Figura 4
Modelo de datos simplificado de MetaFit



Diagrama simplificado con las entidades principales. El esquema completo, con sus 25 tablas, está en los scripts de `database/`. RESTRICCION excluye ejercicios y alimentos mediante tablas de exclusión.

5.4 Flujo de información

1. El cliente (web o móvil) envía la petición HTTP con el token JWT.
2. El middleware valida el token y el rol del usuario.
3. El controlador valida los datos y llama al servicio.
4. El servicio aplica la lógica de negocio y consulta MariaDB mediante el modelo.
5. La API responde en JSON.

En segundo plano, un cron revisa cada hora los pagos por vencer y envía recordatorios por correo, y n8n ejecuta automatizaciones de factura, recordatorio de pago, Telegram y Google Sheets.

6. Requisitos del entorno

6.1 Requisitos de hardware

Tabla 9
Requisitos de hardware

Recurso	Mínimo	Recomendado
CPU	2 núcleos	4 núcleos
RAM	4 GB	8 GB
Disco	10 GB libres	20 GB libres (SSD)
Red	Conexión a Internet estable	Banda ancha estable con acceso a Cloudinary, Brevo y Expo Push

6.2 Requisitos de software

Backend: Node.js 22 (imagen `node:22-alpine`; el CI de GitHub usa Node 20) y npm. Base de datos: MariaDB 11. Contenedores: Docker y Docker Compose. Web: navegador moderno. Móvil: Expo SDK 55 y EAS CLI para generar el APK. Pruebas: Jest (backend y móvil) y Vitest (web).

6.3 Dependencias externas

- Cloudinary: fotos de perfil; si no se configura, el backend guarda en disco local (`backend/uploads`).
- Brevo: correos de recuperación de contraseña, bienvenida, factura y recordatorio de pago.
- Expo Push Service: notificaciones push a la app móvil.
- n8n: automatizaciones (puerto 5678).
- Render y Railway: alojamiento del backend y de la base de datos.
- Google Analytics 4 y Tag Manager: analítica del frontend.

7. Configuración

7.1 Archivos de configuración

Tabla 10
Archivos de configuración

Archivo	Ubicación	Función
`.env`	Raíz del repositorio (copia de `.env.example`)	Variables del backend y de Docker
`docker-compose.yml`	Raíz	Servicios locales: MariaDB, backend, frontend, phpMyAdmin y n8n
`render.yaml`	Raíz	Despliegue del backend en Render
`backend/Dockerfile`	`backend/`	Imagen del backend (Node 22 alpine, puerto 3001)
`frontend_web/.env.development`	`frontend_web/`	URL de la API para el frontend
`movil/app.json` y `movil/eas.json`	`movil/`	Configuración de Expo y de los builds EAS
`.github/workflows/ci.yml`	`.github/workflows/`	Pipeline de CI/CD

7.2 Variables de entorno y parámetros

Tabla 11
Variables de entorno y parámetros

Parámetro	Descripción	Valor por defecto
`PORT`	Puerto del backend	3001
`NODE_ENV`	Entorno de ejecución	`production` en Render
`DB_HOST`	Servidor de la base de datos	`db` en Docker, `localhost` fuera de Docker
`DB_PORT`	Puerto de la base de datos	3306
`DB_USER`	Usuario de la base de datos	`root`
`DB_PASSWORD`	Contraseña de la base de datos	[definir, no publicar]
`DB_NAME`	Nombre de la base	`metafit`
`DATABASE_URL`	Cadena de conexión completa (Render y Railway)	`mysql://usuario:clave@host:3306/metafit`
`JWT_SECRET`	Secreto para firmar los tokens	[generar uno propio de 64 caracteres o más]
`JWT_EXPIRES_IN`	Duración del token	`8h`
`CORS_ORIGINS`	Orígenes permitidos, separados por coma	`localhost:5173` y `localhost:8081`
`CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`	Fotos de perfil	[credenciales propias]

Parámetro	Descripción	Valor por defecto
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`	Envío de correo (Brevo)	[credenciales propias]
`VITE_API_URL`	URL de la API que usa el frontend	`http://localhost:3001`

7.3 Usuarios, roles y permisos

El acceso se controla con JWT y cuatro roles:

Tabla 12
Roles y accesos

Rol	Acceso
Administrador	Web: dashboard, finanzas, afiliados, rutinas, dietas y personal
Entrenador	Web: rutinas, dietas y afiliados (solo lectura)
Recepcionista	Web: afiliados (alta y edición) y pagos
Afiliado	Móvil: perfil, rutina, dieta y progreso

Los usuarios de prueba que carga `04_datos_iniciales.sql` deben cambiarse o eliminarse antes de pasar a producción.

8. Estructura del código fuente

8.1 Organización de carpetas

```
Equipo_Metafit/
├── backend/          API REST (Node.js + Express)
│   ├── routes/ controllers/ services/ models/
│   └── middlewares/ config/ cron/ migrations/ templates/ utils/ __tests__/
├── frontend_web/     Panel web (React + Vite)
│   └── src/          views, components, context, services, utils
├── movil/            App de afiliados (React Native + Expo)
│   └── src/          screens, components, context, navigation, services,
hooks
├── database/         Scripts SQL 01 a 05
├── documentacion/    Manuales, diagramas y auditorías
├── postman/          Colecciones de la API
├── n8n/flujos/       Flujos de automatización
├── docker-compose.yml, Dockerfile, render.yaml
└── .github/workflows/ci.yml
```

8.2 Estándares de codificación

El backend sigue MVC con capa de servicios: las rutas definen los endpoints, los controladores reciben la petición, los servicios contienen la lógica de negocio y los modelos ejecutan el SQL con mysql2, sin ORM. Los archivos usan camelCase con el sufijo de su capa, por ejemplo `afiliadoController.js` y `afiliadoModel.js`. El código se versiona con Git en GitHub y el CI corre en `main` y en la rama `feature/juan-carvajal`.

8.3 Componentes y clases principales

- `backend/middlewares/auth.js`: valida el JWT y restringe el acceso por rol.
- `backend/services/authService.js`: firma los tokens y aplica bcrypt a las contraseñas.
- `backend/controllers/afiliadoController.js` y `services/afiliadoService.js`: alta y gestión de afiliados.
- `backend/controllers/planController.js`: planes de entrenamiento y nutrición.
- `backend/controllers/pagoController.js` y `services/facturaService.js`: pagos y factura por correo.
- `backend/cron/recordatorioPagos.js`: recordatorios de pago cada hora.
- `backend/services/pushService.js`: notificaciones push con Expo.
- `backend/config/db.js`: conexión a la base de datos; `config/swagger.js` y `config/cloudinary.js`.
- `frontend_web/src/views`: pantallas del panel, como `RutinasView.jsx`.
- `movil/src/screens`: pantallas de la app, como `MiRutinaScreen.js`, `MiDietaScreen.js` y `MiProgresoScreen.js`.

9. Base de datos

9.1 Motor y versión

MariaDB 11, compatible con MySQL y usada igual en Docker y en producción. La base se llama `metafit`.

9.2 Scripts de creación y migración

Los scripts están en `database/` y se ejecutan en orden numérico, porque el 04 inserta en tablas creadas por el 02 y el 03.

Tabla 13
Scripts de la base de datos

Orden	Archivo	Qué hace
1	`01_estructura.sql`	Tablas base y vistas
2	`02_migracion_movil.sql`	Tablas de la app móvil: progreso diario, agua y consumo de alimentos
3	`03_mejoras_estructura.sql`	Columnas nuevas de AFILIADO y tablas de registro de ejercicio y consumo (idempotente)
4	`04_datos_iniciales.sql`	Datos iniciales: personal, afiliados, catálogos y planes (idempotente)
5	`05_password_reset.sql`	Tabla PASSWORD_RESET para recuperar contraseñas

Docker los ejecuta solo en la primera inicialización del volumen. Para repetirlo: `docker compose down -v && docker compose up -d`. Además, el backend corre migraciones JavaScript idempotentes al arrancar (`backend/migrations`).

9.3 Diccionario de datos

Tabla 14
Diccionario de datos

Tabla	Campos principales	Descripción
USUARIO	id_usuario, nombres, apellidos, correo, contrasena, rol, estado	Cuentas de todos los roles; la contraseña se guarda con hash bcrypt
AFILIADO	id_usuario, documento, fecha_nacimiento, sexo, telefono, estatura_cm	Datos personales del afiliado, ligados a USUARIO

Tabla	Campos principales	Descripción
CICLO	id_ciclo, id_usuario, fecha_inicio, fecha_fin, activo, objetivo_fisico	Periodo de entrenamiento de un afiliado
RUTINA	id_rutina, id_ciclo, nombre_rutina, enfoque_muscular, dia_numero	Rutina de un día del ciclo
RUTINA_EJERCICIO	id_rutina, id_ejercicio, orden, series, repeticiones, peso_kg, descanso_seg	Ejercicios de cada rutina
PLAN_NUTRICIONAL	id_ciclo, calorias_objetivo, num_comidas	Plan de alimentación del ciclo
DETALLE_NUTRICIONAL	id_ciclo, num_comida, id_alimento, cantidad_g	Alimentos de cada comida
EJERCICIO	id_ejercicio, nombre_ejercicio, grupo_muscular, nivel_minimo	Catálogo de ejercicios
ALIMENTO	id_alimento, nombre_alimento, proteinas, carbohidratos, grasas	Catálogo de alimentos
RESTRICCION	id_restriccion, nombre_restriccion, tipo	Lesiones, alergias y enfermedades que excluyen ejercicios o alimentos
PROGRESO_FISICO	id_ciclo, fecha_registro, peso_kg, porcentaje_grasa	Evolución física de un ciclo
PAGO	id_pago, id_usuario, fecha_pago, valor_pagado, estado, fecha_vencimiento	Membresías; estado Pagado, Vencido o Pendiente
CONFIGURACION	clave, valor	Parámetros del sistema, como el precio de la membresía

10. Interfaces e integraciones (APIs)

Tabla 15
Endpoints de la API

Endpoint	Métodos	Descripción
`/login`	POST	Inicio de sesión (máximo 10 intentos cada 15 minutos)
`/auth/recuperar-password`, `/auth/reset-password`	POST	Recuperación de contraseña por correo
`/usuarios`, `/usuarios/:id`	GET, POST, PATCH, DELETE	Usuarios del personal; también `/usuarios/recepcionistas` y `/usuarios/me/push-token`
`/afiliados`, `/afiliados/:id`	GET, POST, PATCH, DELETE	CRUD de afiliados
`/afiliados/me`	GET, PATCH	Perfil del afiliado autenticado
`/afiliados/me/foto`, `/afiliados/:id/foto`	POST	Foto de perfil
`/afiliados/:id/ciclos`, `/afiliados/ciclos`	GET, POST	Ciclos de entrenamiento

Endpoint	Métodos	Descripción
`/afiliados/:id/restricciones`	GET, POST, DELETE	Restricciones médicas del afiliado
`/afiliados/:id/pagos`	GET, POST	Pagos de un afiliado
`/afiliados/me/...`	GET, POST	Seguimiento diario: ejercicio, agua, consumo de alimentos y notas
`/pagos`, `/pagos/metricas`	GET	Listado y métricas de pagos
`/planes/entrenamiento`, `/planes/rutinas`	GET, POST, PATCH, DELETE	Planes de entrenamiento y rutinas
`/planes/nutricional`	GET, POST, PATCH, DELETE	Planes nutricionales y su detalle
`/catalogo/ejercicios`, `/alimentos`, `/restricciones`	GET, POST, PUT, DELETE	CRUD de catálogos
`/ciclos/:id_ciclo`	PATCH, DELETE	Editar o eliminar un ciclo
`/progreso/resumen`, `/historial`, `/ejercicio/:id/evolucion`	GET, PUT	Progreso del afiliado
`/dashboard/kpis`	GET	Indicadores del gimnasio
`/configuracion/precio-membresia`	GET, PUT	Precio de la membresía
`/notificaciones`	GET	Notificaciones
`/health`, `/api-docs`	GET	Estado del servicio y documentación Swagger

Las rutas protegidas requieren el encabezado `Authorization: Bearer <token>` (JWT de 8 horas); `/login`, la recuperación de contraseña y `/health` son públicas. Los parámetros y respuestas de cada endpoint están en Swagger (`/api-docs`) y en la carpeta `postman/`.

11. Seguridad

11.1 Autenticación y autorización

La API usa tokens JWT firmados con `JWT_SECRET` y válidos por 8 horas. Los middlewares de `backend/middlewares/auth.js` restringen cada ruta por rol (Administrador, Entrenador, Recepcionista) y las rutas `/afiliados/me` sirven al afiliado autenticado. Las contraseñas se guardan con bcrypt de 12 rondas, con un máximo de 72 bytes. El login admite 10 intentos cada 15 minutos.

11.2 Protección de datos y cifrado

El backend usa Helmet para las cabeceras de seguridad, limita el cuerpo de las peticiones a 50 KB y valida el Content-Type. CORS se configura con `CORS_ORIGINS`. Pendiente conocido: CORS acepta hoy cualquier origen (`origin: '*'`) y el roadmap prevé reactivar la lista blanca.

11.3 Gestión de registros (logs) y auditoría

El backend escribe en la salida estándar: en Render se consulta en el panel del servicio y en local con `docker compose logs`. La disponibilidad se vigila con UptimeRobot (ver apartado 14.2).

11.4 Cumplimiento normativo

Las contraseñas nunca se almacenan en texto plano. El sistema se evaluó contra ISO/IEC 25010 (`documentacion/auditoria_iso_25010_FINAL.md`). Pendiente de definir: la política de tratamiento de datos personales aplicable en Colombia.

12. Pruebas

12.1 Plan de pruebas

Hay 74 pruebas automáticas: 25 en el backend, 30 en el frontend y 19 en la app móvil. Se ejecutan con `npm test` en cada carpeta y en GitHub Actions en cada push. Las pruebas de la API también se pueden hacer a mano con las colecciones de `postman/`. El informe de QA está en `documentacion/QA_REPORT.md`.

12.2 Casos de prueba y resultados

Tabla 16
Pruebas automáticas por módulo

Módulo	Herramienta	Pruebas	Comando	Estado
Backend	Jest y Supertest	25	`cd backend && npm test`	En verde según el README
Frontend web	Vitest y Testing Library	30	`cd frontend_web && npm test`	En verde según el README
App móvil	Jest y jest-expo	19	`cd movil && npm test`	En verde según el README

12.3 Incidencias conocidas

Según el roadmap, quedan pendientes dos puntos: restringir CORS (hoy abierto a cualquier origen) y documentar todas las variables en `.env.example`. El historial de errores resueltos está en `documentacion/brain/Bugs y soluciones/Historial de bugs.md`.

13. Despliegue y entrega

13.1 Entornos

Local con Docker Compose y producción con el backend en Render (Docker), el frontend web en Render (sitio estático), la base de datos en Railway y la app móvil en EAS. No hay un entorno de preproducción documentado.

13.2 Procedimiento de despliegue

1. Subir los cambios a una rama y abrir un pull request hacia `main`: el CI ejecuta las pruebas del backend, del frontend (con build) y de la app móvil.
2. Al llegar a `main`, el job de CD llama a la API de Render y despliega el backend y el frontend. Necesita el secreto `RENDER_API_TOKEN` en GitHub.
3. Render construye la imagen de `backend/Dockerfile` y comprueba `/health`.
4. Verificar `/health` y `/api-docs` en producción.

5. App móvil: generar el build con EAS según `documentacion/manual_apk_movil.md`.

   Las variables de producción se configuran en el panel de Render: `NODE_ENV`, `DATABASE_URL`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `CORS_ORIGINS` y `PORT`.

13.3 Plan de contingencia y retorno (rollback)

El proyecto no documenta un plan de retorno. La opción disponible es revertir el commit en `main`, lo que dispara un nuevo despliegue. Antes de cambios en la base de datos, haz una copia (ver apartado 9.4).

13.4 Lista de entregables

Tabla 17
Lista de entregables

Entregable	Formato o ubicación	Entregado (Sí/No)
Código fuente	Repositorio Git: `backend`, `frontend_web`, `movil`	
Manual técnico	Este documento	
Manual de usuario		
Scripts de base de datos	`database/01` a `05`	
Colecciones Postman	`postman/`	
Flujos n8n	`n8n/flujos/`	
APK de la app móvil	Build de EAS	
Acta de entrega	Por redactar	

14. Mantenimiento y soporte

14.1 Tareas de mantenimiento periódico

- Cada hora, el cron `backend/cron/recordatorioPagos.js` envía por correo un recordatorio de los pagos que vencen en los próximos 3 días, sin repetirlo el mismo día.
- Recomendado: revisar los logs, actualizar dependencias con `npm audit` y ejecutar las pruebas antes de cada entrega.

- Renovar las credenciales de Cloudinary y Brevo cuando caduquen.

14.2 Monitorización y rendimiento

UptimeRobot vigila la disponibilidad del backend (`documentacion/UPTIME_ROBOT.md`), Code Climate mide la calidad del código (`.codeclimate.yml`) y Google Analytics 4 con Tag Manager registra el uso del frontend.

14.3 Gestión de incidencias y niveles de soporte

Las incidencias se llevan en Jira (`documentacion/INFORME_JIRA_METAFIT.md`) y los errores resueltos se anotan en `documentacion/brain/Bugs y soluciones/Historial de bugs.md`. Los niveles de soporte están pendientes de definir.

14.4 Procedimiento de actualización

1. Hacer una copia de la base de datos.
2. Integrar los cambios por pull request a `main`.
3. Esperar al CI y al despliegue automático en Render.
4. Si cambia el esquema, crear una migración idempotente en `backend/migrations`.
5. Verificar `/health` y `/api-docs`.

14.5 Contactos de soporte

Tabla 18
Contactos de soporte

Nombre	Rol	Correo	Teléfono
Isabella Caldas Perez		isabellacaldasperez@gmail.com	3778255691

15. Solución de problemas (FAQ técnica)

Tabla 19
Problemas frecuentes y soluciones

Problema / Mensaje de error	Causa probable	Solución
El backend no conecta con la base de datos	`DB_HOST` incorrecto (en Docker es `db`, fuera de Docker `localhost`) o MariaDB aún iniciando	Revisar `.env` y `docker compose ps`
Error 429 al iniciar sesión	Se superaron 10 intentos en 15 minutos	Esperar 15 minutos e intentar de nuevo
La sesión deja de ser válida	El token JWT dura 8 horas	Iniciar sesión otra vez
Faltan los datos de prueba tras `docker compose up`	Los scripts SQL solo corren en la primera inicialización del volumen	`docker compose down -v` y levantar de nuevo
`04_datos_iniciales.sql` falla	Se ejecutó antes que el 02 o el 03	Ejecutar los scripts en orden del 01 al 05
Las fotos de perfil se pierden al reiniciar	Cloudinary no está configurado y se guardan en disco local	Definir las variables `CLOUDINARY_*`
No llegan los correos	Credenciales SMTP de Brevo ausentes o incorrectas	Revisar las variables `SMTP_*`

16. Glosario

Tabla 20
Glosario de términos

Término	Definición
API REST	Interfaz que permite a la web y a la app pedir y enviar datos al servidor por HTTP
bcrypt	Algoritmo para guardar contraseñas como hash, no en texto plano
CI/CD	Integración y despliegue continuos: pruebas y publicación automáticas en cada cambio
CORS	Regla del navegador que define qué orígenes pueden llamar a la API
Cron	Tarea programada que se ejecuta cada cierto tiempo
Docker	Herramienta que empaqueta una aplicación en contenedores para ejecutarla igual en cualquier equipo
EAS	Expo Application Services, servicio que compila la app móvil
Idempotente	Script que se puede ejecutar varias veces sin duplicar datos ni fallar
JWT	Token firmado que identifica al usuario en cada petición
MVC	Patrón que separa modelo, vista y controlador
ORM	Capa que traduce objetos a SQL; MetaFit no la usa
RBAC	Control de acceso según el rol del usuario
Swagger	Documentación interactiva de la API, disponible en `/api-docs`

Referencias

Ministerio de Tecnologías de la Información y las Comunicaciones. (2018). *G.SIS.03 Guía para la construcción del catálogo de sistemas de información*. https://mintic.gov.co/arquitecturati/630/w3-article-75551.html

17. Anexos

- Anexo A: diagramas de flujo de autenticación, asignación de rutina y consulta de perfil, en **`documentacion/DIAGRAMAS.md`** (secciones 3.5 a 3.7). Los diagramas de arquitectura, despliegue y datos están en el apartado 5.
- Anexo B: scripts SQL en **`database/`** y flujos de n8n en `n8n/flujos/`.
- Anexo C: acta de entrega y aceptación (documento aparte, por redactar).
- Anexo D: referencias: **`documentacion/manual_despliegue.md`**, **`documentacion/MANUAL_POSTMAN.md`** y **`documentacion/auditoria_iso_25010_FINAL.md`**.